Experiment 6: Merge Sort

Aim

To implement the **Merge Sort** algorithm using Python to sort a list of elements in ascending order.

Algorithm

Step 1: Start.

Step 2: Read the number of elements and the array.

Step 3: Divide the array into two halves.

Step 4: Recursively sort the left half.

Step 5: Recursively sort the right half.

Step 6: Merge the two sorted halves into a single sorted array.

Step 7: Display the sorted array.

Step 8: Stop.

Source Code (Python)

main.py

```python
# Merge Sort

def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2

        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

n = int(input("Enter number of elements: "))

arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

merge_sort(arr)

print("Sorted array:", arr)
```

Output

```
Output

Enter number of elements: 6
Enter the elements:
45
10
20
80
50
30
Sorted array: [10, 20, 30, 45, 50, 80]

=== Code Execution Successful ===
```

Time Complexity

- **Best Case:** O(n log n)
- **Average Case:** O(n log n)
- **Worst Case:** O(n log n)

Space Complexity

- **O(n)**

Result

The **Merge Sort** algorithm was implemented successfully in Python. The program sorted the given list of elements in ascending order using the **Divide and Conquer** technique, and the output matched the expected result.